Data-Driven VMS-LES Modeling

An Overview of its History & Future

M.Sc. Thesis: Research Proposal

Maxim Michielsen

Delft University of Technology

TU Delft

Data-Driven VMS-LES Modeling

An Overview of its History & Future

by

Maxim Michielsen

Instructor:	Dr. S.J. Hulsoff
Project Duration:	Month, Year - Month, Year
Faculty:	Faculty of Aerospace Engineering, Delft

Cover:	FILL IN
Style:	TU Delft Report Style, with modifications by Daan Zwaneveld and Maxim Michielsen

TU Delft

Contents

Nomenclature ii

1 Introduction 1

2 Problem Description 3

3 Literature Review 4

3.1 Computational Fluid Dynamics 4

3.1.1 Variational Multiscale Framework 5

3.2 Artificial Intelligence 6

3.2.1 Artificial Neural Networks 6

3.2.2 Multilayer Perceptron Models 7

3.3 Machine Learning 8

3.3.1 Supervised & Unsupervised Learning 8

3.3.2 Reinforcement Learning 9

3.4 CFD-ANN Enriched Models 15

3.4.1 Other work 15

3.4.2 Past Work at TU Delft 15

4 Research Plan 18

4.1 Proposal 18

4.2 Project Plan 19

References 21

A MLP Extensive Explanation 23

Nomenclature

Abbreviations

AC3 Asynchronous Advantage Actor–Critic.

AI Artificial Intelligence.

ANN Artificial Neural Network.

AV Artificial Viscosity.

CFD Computational Fluid Dynamics.

CFL Courant-Friedrichs-Lewy.

CNN Convolutional Neural Network.

CPI Corrector-Pass Instability.

CQL Conservative Q-Learning.

DNS Direct Numerical Simulation.

FEM Finite Element Method.

GAV Global Artificial Viscosity.

IQL Implicit Q-Learning.

KL Kullback-Leibler.

LAV Local Artificial Viscosity.

LES Large Eddy Simulation.

LFS Lagged Feature Set.

LTI Long-Term Instability.

ML Machine Learning.

MLP Multi-Layer Perceptron.

OOD Out-of-Distribution.

PDE Partial Differential Equation.

PPO Proximal Policy Optimization.

RANS Reynolds-Averaged Navier-Stokes.

RL Reinforcement Learning.

RNN Recurrent Neural Network.

SAC Soft Actor–Critic.

SGS Subgrid-Scale.

TD3 Twin Delayed Deep Deterministic Policy Gradient.

TD3+BC Twin Delayed Deep Deterministic Policy Gradient with Behavior Cloning.

TRPO Trust Region Policy Optimization.

VMS Variational Multiscale.

1

Introduction

Check Draft

The Navier-Stokes equations describe the motion of viscous fluid substances. Although they are universally applicable to fluids, a mathematical solution to the equations is yet to be derived, and they remain as one of the millennium prize problems. Due to its complex nature, engineers instead opt to model different parts of the equations, depending on the nature of the problem they wish to simulate. For Computational Fluid Dynamics (CFD), several approaches exist, ranging in physical fidelity and computation cost, from the simplest Reynolds-Averaged Navier-Stokes (RANS) to the Direct Numerical Simulation (DNS) of fluids. DNS scales rapidly with the Reynolds Number of the flow, computationally infeasible for most projects ...

Over the years, computational power increased exceptionally fast. However, as processors are starting to reach the atomic scale, it is evident that this rapid growth of computational power might come to a halt. It is due to this expected slowing down of computation power, new, innovative methods for simulating the NS equations are needed. For Large Eddy Simulation (LES), new approaches of coupling data-driving closure models to the numerical solver are being tested. Using projected DNS data, these data models might be able to learn the underlying physics, and become a universal model for subgrid-scale closure modeling. If successful, LES can become more practical and accurate, whilst being computationally cheaper than DNS. Ultimately, LES can be used to inform RANS in a similar fashion, improving RANS accuracy and allowing for more robust, yet relatively very cheap, simulations of fluids.

Coupling an Artificial Neural Network (ANN) to a Variational Multiscale (VMS) formulation has been attempted prior. First, Robijns [12] laid the groundworks and established the fact that strong *a priori* correlations between predicted ANN closures and projected DNS closures can be achieved. However, later Janssens [5] showed that while strong a priori correlations are possible, these are not directly translated to *a posteriori* performance. In fact, using the ANN predictions led to two critical instability mechanisms: Corrector-Pass Instability (CPI) and Long-Term Instability (LTI). Rajampeta [11] was able to remove the CPI from the Newton corrector passes by the use of a

Lagged Feature Set (LFS), and used backscatter limiting to address the LTIs. He was successful in achieving a stable online run, but backscatter limiting is unphysical in its nature, motivating future work for discovering other methods which could address the LTI problem. Bettini [1] attempted a more energy-constrained approach, which led to over dissipative simulation runs. Krochak [7] continued the work, attempting three different stabilization strategies, yet the LTI still remained. Dominik...

cite dominik

From previous work, it is clear that the problem of LTIs is detrimental, yet persistent. For this work, a new approach of including an additional artificial dissipation term is taken. This artificial term is intended to correct the erroneous predictions of the model, and to drain the accumulating energy, in order to hopefully bring balance to the force.

Check the goal of this artificial dissipation term, is this how we are gonna use it for?

Continuing the persistent problem of solver instability when using VMS-ANN coupled models, the research proposal is defined as follows.

Research Proposal

Research Plan

In order to further specify the proposal, the following research questions are formulated:

1. **Question 1**
   - 1.a Subquestion 1.1
   - 1.b ...
2. **Question 2**
3. **...**

2

Problem Description

3

Literature Review

3.1. Computational Fluid Dynamics

add citations where needed

CFD simulations are used to solve various fluid dynamic problems. An example of applying CFD simulations is the Navier-Stokes equations. These equations govern the flow of fluids over bodies (or without them) and are thus important in many engineering practices, from aeronautical to automotive. Unfortunately, no analytical solution exists, so engineers must approximate these equations numerically. The most accurate numerical method is DNS. Here, the solver resolves all flow scales, yielding the most accurate results but at a high computational cost. In fact, representing all scales increases computational costs very quickly. The smallest scales, also known as the Kolmogorov microscales, are all proportionate to the Reynolds number, Re. For three-dimensional DNS, costs are proportionate to Re^3. Thus, for large-scale flow problems, DNS is computationally infeasible, and it is currently used primarily as a research tool for studying turbulent phenomena.

To reduce computational costs, researchers have derived a method that splits the solution into a mean and a fluctuating part. This method, called RANS, is among the least expensive simulations in the CFD landscape. Here, the solver computes the mean solution while accounting for fluctuations through modeling. Despite various turbulence models, RANS simulations struggle to incorporate the influence of the fluctuating flow components on the mean, especially for highly dynamic and wall-bounded problems.

A third method of solving fluid dynamics problems is to use LES. This method lies between the low-fidelity, cheap RANS simulations and the high-fidelity, expensive DNS simulations. Similarly to DNS, the solver resolves the flow scales, but only the largest, most influential ones. LES is thus a DNS, with a spatial filter applied, which removes the smaller, and more expensive scales. This approach makes LES much cheaper than DNS, but it does remove some physical fidelity. Naturally,

if greater physical accuracy is required, the filter width can be reduced, allowing LES to approach and eventually become DNS.

This approach splits up the solution into the larger scales, also known as the resolved scales, and the smaller, Subgrid-Scale (SGS) terms. The solver models the influence of SGS terms. Modeling is necessary because smaller scales serve two important functions. One, they drain the flow field of energy through dissipative phenomena. Two, in specific scenarios, they re-transfer energy to the larger scales through a process known as *backscatter*. The standard modeling approach is to use the Smagorinsky model. This pioneering model is relatively simple and correctly captures energy cascades in simulations, making it a great option. However, the model has several drawbacks, including limited wall behavior, no backscatter, and a tendency to be overly dissipative. Other models exist, such as the Dynamic Smagorinsky and Scale Similarity models.

The problem with classical LES is that the solver applies the influence of the model to the complete flow domain, which tends to make simulations overly dissipative. A different approach, developed by Hughes, uses the variational multiscale framework.

3.1.1. Variational Multiscale Framework

add citations where needed

VMS is an unconventional approach to LES. Instead of applying a low-pass spatial filter, the approach performs a finite-element projection, which splits the continuous unknown into a resolvable component and a subgrid component. This projection naturally introduces the spatial filter. The framework has several advantages over a classical LES formulation. First, the effect of modeling is confined only to the smallest resolved scales, which can reduce the effect of modeling errors. Second, with a stronger mathematical foundation, it can perform better in wall-bounded geometries.

Consider a general nonlinear Partial Differential Equation (PDE) written in residual form:

$$\mathcal{R}(u) = 0 \tag{3.1}$$

where $u \in \mathcal{V}$ is the solution in a suitable function space (e.g. Sobolev space) and $\mathcal{R}(u)$ is the residual operator. The variational formulation reads:

Find $u \in \mathcal{V}$ such that

$$(\mathcal{R}(u), w) = 0 \quad \forall w \in \mathcal{V} \tag{3.2}$$

where w are test functions and $(.,.)$ denotes an inner product over the domain. VMS decomposes the solution space into

$$\mathcal{V} = \bar{\mathcal{V}} \oplus \mathcal{V}' \tag{3.3}$$

Where $\bar{\mathcal{V}}$ are the resolved (coarse) scales and $\mathcal{V}'$ are the unresolved (fine/subgrid) scales. Correspondingly,

$$u = \bar{u} + u' \qquad w = \bar{w} + w' \tag{3.4}$$

Substituting this decomposition back into the weak form yields two coupled variational equations: a coarse-scale equation and a fine-scale equation. The fine-scale equation can be substituted into the coarse-scale equation, yielding the final form. Solvers must approximate the contribution from the fine-scale equation.

3.2. Artificial Intelligence

add citations where needed

Artificial Intelligence (AI) is the simulation of human intelligence by computer systems to perform tasks like learning, reasoning, problem-solving, and decision-making, and is currently rapidly evolving due to recent breakthroughs. Applications of AI are extensive and diverse, ranging from interpreting patterns in speech to analyzing medical X-rays to detect anomalies, etc. Using Machine Learning (ML), they train on vast amounts of data. ML enables systems to automatically learn, analyze patterns, and improve from data without being explicitly programmed. This section explains ANNs plainly, as well as giving a basic overview of ML and the Multi-Layer Perceptron (MLP).

3.2.1. Artificial Neural Networks

add citations where needed

ANNs are humanity's best attempt at creating a digital analogy to the human brain. As the human brain is capable of performing complex tasks, such as recognizing faces in an instant, it is no wonder that humans are motivated to create models that mimic its behavior, and hopefully, its performance. Describing the human nervous system can, in essence, be done as a three-block system: receptors, effectors, and, centrally, the neural net, which is composed of neurons. Although neurons are much slower than silicon logic gates, the brain can compensate by having an enormous number of densely interconnected neurons. It is this dense interconnected neural network that inspired the mathematical models used to create ANNs.

Inspired by the human brain, ANNs can be described as systems of parallel, distributed proces-

sors, each composed of simple processing units. It acquires knowledge through a learning process in which it interacts with its environment and adapts the strengths of its interneuron connections (synaptic weights). Many types of ANN models exist using this definition. ANN is thus an umbrella term encompassing various models, each with its own advantages and disadvantages.

One of the simplest types of models is the feed-forward neural network, also called the MLP. Other than MLPs, well-known types of models are Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN) models. CNNs are used for tasks such as predicting data types, including text, images, and audio. By applying filters, they create feature maps that can detect patterns such as edges, lines, and textures. RNNs on the other hand, use internal loops to retain memory of previous inputs. This approach makes them better suited for language translation and speech recognition. Of course, many other types of models exist, as the AI field is currently rapidly evolving.

As AI models are statistical approximators, they require large amounts of data for training. Not only that, but data has to be split up into three components: the actual training set, a validation set, and a testing set. The actual training set is indeed the dataset the model uses for learning and parameter updates. Next, the validation set decides when training should halt. It does not influence the weights; instead, it evaluates the model's current performance during training. The last dataset, the test set, is used once after training is complete. It evaluates the model's true generalization performance. The model must never see this set during training or tuning.

Indeed, the goal of the model is generalization, meaning it can perform well on similar datasets not seen during training. The opposite of generalization is overfitting. When a model is overfit, it only performs well on the training dataset. The model may have learned the underlying relationship, but it may also have learned the noise in the training dataset. It has memorized the training samples instead of capturing the true structure. Causes of overfitting can include too little data or too many training epochs. Common solutions include early stopping, regularization, and dropout.

3.2.2. Multilayer Perceptron Models

add citations where needed

MLP, also known as feed-forward neural networks, are one of the simplest frameworks in the ANN landscape. They consist of three main layers: the input layer, the output layer, and the hidden layer(s) in between. Each of these layers contains an arbitrary number of neurons, and each neuron in one layer connects with every node in the following layer. Their connections are mathematically presented by

$$z = w^T x + b \tag{3.5}$$

$$a = \sigma(z) \tag{3.6}$$

where x is the input vector, w are the weights, b is a bias term, and z represents the linear combination of the input, weights, and bias. z is then passed on to an activation function, σ, which is typically a sigmoid, tanh, or ReLU. a is the final neuron output, which becomes the input for the next layer of neurons. The input signal thus propagates forward through the model, passing each hidden layer until reaching the output layer.

$$x \to a^{(1)} \to a^{(2)} \to \ldots \to \hat{y}$$

During training, comparing the predicted value $\hat{y}$ to the true value y provides an error signal that propagates backward through the model. During backpropagation, the error contributed to each weight is computed, providing a measure of how to adjust it. One forward-backward loop is called an epoch. After many epochs, training stops when the stopping criteria are met.

As they are the simplest types of models, they will be used as the baseline model for predicting the SGS closure terms, as explained in Chapter 2.

3.3. Machine Learning

add citations where needed

ANNs are trained on huge amounts of data to approximate underlying functions, recognizing data structures, etc. These models need to be informed of the accuracy of their predictions, providing the signals required for backpropagation. There are mainly three learning paradigms: supervised learning, unsupervised learning, and reinforcement learning. Supervised learning requires a teacher to provide the models with correct target responses. On the contrary, unsupervised learning has no teacher. Instead, models train to discover and classify data structures. Lastly, reinforcement learning does not have a teacher present during learning; however, it differs from unsupervised learning in that the model still has to predict a continuous value. Reinforcement learning is thus a distinct learning philosophy, making it incorrect to classify it as supervised or unsupervised learning.

3.3.1. Supervised & Unsupervised Learning

add citations where needed

Supervised learning is currently the most studied machine learning paradigm, thanks to its straightforward nature. The main distinction from other paradigms is the presence of a teacher.

In other words, the availability of labeled training examples. Each example consists of a unique input signal and a corresponding desired response (target). The network's parameters are adjusted to minimize the error signal between the input and the target. This learning paradigm typically applies to regression problems, where the model predicts continuous values from input data.

Unsupervised learning is an approach in which a model learns patterns from unlabeled data. Typical objectives include grouping similar data, reducing dimensionality, or finding underlying features or structures. These objectives are called classification problems.

3.3.2. Reinforcement Learning

add citations where needed

Reinforcement Learning (RL) consists of an interaction loop between an *agent* and an *environment*. The agent decides what actions to take, based on its current state, which it receives from the environment. The environment can be more abstract than just the outside world. For example, say an agent is a robot with a battery. Even though the battery is internal to the robot, the charge status should still be considered part of the environment by the agent and influence the agent's decisions. Besides the agent and the environment, RL consists of four main sub-elements: a policy, a reward signal, a value function, and, optionally, a model of the environment. A policy defines the actions the agent will undertake given its current state and, roughly speaking, can be seen as a mapping from the agent's current state to its actions. A reward signal is the instantaneous goal of a learning problem. Usually, it is a scalar signal received at every step the agent takes. Agents will need to balance instantaneous rewards with long-term returns. The value function measures how beneficial it is for an agent to be in a particular state, taking into account the future rewards it can still obtain. Finally, the model of the environment describes how the environment behaves when the agent takes an action. It predicts the next state and the associated reward. Reinforcement learning splits into two methods: model-based and model-free, depending on whether a model is available. Of the two, model-free is more relevant to the problem defined in Chapter 2.

Reinforcement learning contains several sub-methods for calculating rewards, updating policies, etc. Ranging from simpler tabular methods to more complex methods for continuous domain problems. For the research proposal, the latter methods are more relevant as the state spaces is tremendous, and offline off-policy learning is expected to yield the best performance while remaining computationally feasible.

Rewrite the gpt stuf!

Online vs. Offline Reinforcement Learning Reinforcement learning algorithms can broadly be categorized into *online* and *offline* methods depending on how training data is obtained. In online reinforcement learning, the agent learns by continuously interacting with the environment and

collecting new data during training. The policy is therefore updated using trajectories generated by the current policy, allowing the agent to explore the state–action space and improve its behavior over time. Algorithms such as Proximal Policy Optimization (PPO), Soft Actor–Critic (SAC), and fall into this category and have demonstrated strong performance in continuous control and robotics applications.

In contrast, offline reinforcement learning (also known as batch reinforcement learning) assumes that the agent is trained using a fixed dataset of previously collected trajectories without further interaction with the environment. This setting arises in many practical applications where environment interaction is expensive, unsafe, or impossible, such as robotics, healthcare, and scientific simulations. Standard online algorithms often perform poorly in this regime because the learned policy may select actions outside the distribution of the dataset, leading to unreliable value estimates. Consequently, specialized algorithms such as Conservative Q-Learning (CQL), Implicit Q-Learning (IQL), and Twin Delayed Deep Deterministic Policy Gradient with Behavior Cloning (TD3+BC) have been developed to mitigate distribution shift and overestimation errors by constraining policy updates or learning conservative value functions.

Tables 3.1 and 3.2 summarize the main characteristics of the online and offline reinforcement learning algorithms discussed in this work.

Online Algorithms

SAC is an off-policy actor–critic reinforcement learning algorithm based on the maximum entropy reinforcement learning framework. Instead of maximizing only the expected return, SAC optimizes a stochastic policy that maximizes both the expected reward and the entropy of the policy. The objective can be written as

$$J(\pi) = \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[\sum_t r(s_t,a_t) + \alpha H(\pi(\cdot|s_t))\right] \tag{3.7}$$

where $H(\pi(\cdot|s))$ denotes the entropy of the policy and α is the temperature parameter that controls the trade-off between reward maximization and exploration. The entropy term encourages the policy to remain stochastic, which improves exploration and robustness during learning [4].

SAC follows an actor–critic architecture with a stochastic policy (the actor) and two Q-functions (the critics). The Q-functions are trained using a modified Bellman update that incorporates the entropy term:

$$Q(s,a) = r(s,a) + \gamma\mathbb{E}_{s'}\left[Q(s',a') - \alpha\log\pi(a'|s')\right] \tag{3.8}$$

where $a' \sim \pi(\cdot|s')$. Similar to Twin Delayed Deep Deterministic Policy Gradient (TD3), SAC em-

ploys two Q-functions and uses the minimum of the two value estimates to reduce overestimation bias during policy improvement.

The policy is updated by minimizing the objective

$$J_\pi(\phi) = \mathbb{E}_{s\sim D, a\sim\pi_\phi}\left[\alpha \log \pi_\phi(a|s) - Q_\theta(s,a)\right] \tag{3.9}$$

which encourages the policy to select actions that both maximize the expected Q-value and maintain sufficient entropy. In practice, SAC also introduces an automatic temperature tuning mechanism that adjusts α during training to maintain a desired entropy level, which significantly improves stability and reduces the need for manual hyperparameter tuning.

Empirically, SAC has been shown to outperform both on-policy methods such as PPO and earlier off-policy actor–critic algorithms such as DDPG and TD3 on a range of continuous control benchmarks, achieving both improved sample efficiency and higher asymptotic performance [4]. However, SAC is primarily designed for online reinforcement learning settings in which the agent can interact with the environment during training. In purely offline reinforcement learning settings, the entropy-driven policy improvement can lead to significant extrapolation errors when the policy evaluates actions outside the dataset distribution. Consequently, algorithms specifically designed for offline learning, such as CQL or IQL, are generally preferred when training from fixed datasets.

PPO is an on-policy actor–critic reinforcement learning algorithm designed to achieve stable policy updates while maintaining implementation simplicity. PPO belongs to the family of policy gradient methods, which optimize a parameterized stochastic policy by estimating gradients of expected return with respect to policy parameters.

The basic policy gradient estimator is given by

$$\hat{g} = \mathbb{E}_t\left[\nabla_\theta \log \pi_\theta(a_t|s_t)\hat{A}_t\right] \tag{3.10}$$

where $\pi_\theta(a|s)$ is the policy and $\hat{A}_t$ is an estimator of the advantage function [13]. This formulation encourages the policy to increase the probability of actions that yield positive advantages and decrease the probability of actions with negative advantages.

However, performing multiple gradient updates on the same data can lead to excessively large policy updates, resulting in unstable learning. Earlier algorithms such as Trust Region Policy Optimization (TRPO) addressed this by constraining the Kullback-Leibler (KL) divergence between successive policies, but solving this constrained optimization problem is computationally expensive.

PPO simplifies this approach by introducing a clipped surrogate objective that restricts how far the updated policy can deviate from the previous policy. Defining the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \tag{3.11}$$

the PPO objective is defined as

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{3.12}$$

where ϵ is a small hyperparameter (typically around 0.2). The clipping operation prevents the probability ratio from moving too far away from 1, effectively limiting the size of policy updates while still allowing efficient gradient-based optimization.

PPO typically uses an actor–critic architecture in which the policy network (actor) outputs a stochastic action distribution and a value network (critic) estimates the state value $V(s)$. The full objective function combines the clipped policy loss with a value function loss and an entropy bonus to encourage exploration:

$$L(\theta) = \mathbb{E}_t \left[L_t^{\text{CLIP}}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right] \tag{3.13}$$

where L^{VF} is the value function regression loss and $S[\pi]$ denotes the entropy of the policy distribution.

Training proceeds iteratively by collecting trajectories using the current policy, estimating advantages, and then performing several epochs of minibatch stochastic gradient ascent on the surrogate objective before collecting new data. This allows PPO to reuse sampled data efficiently while maintaining stable policy improvement.

Empirically, PPO achieves performance comparable to or better than TRPO while being significantly easier to implement and more computationally efficient. It has become one of the most widely used reinforcement learning algorithms for continuous control and large-scale policy learning problems.

Asynchronous Advantage Actor–Critic (AC3) is an on-policy reinforcement learning algorithm that combines policy gradient methods with value function approximation. The method was introduced to improve the stability and efficiency of deep reinforcement learning by allowing multiple agents to interact with independent copies of the environment in parallel. Each worker maintains its own environment instance and periodically updates a shared set of global network parameters [9].

A3C follows an actor–critic architecture in which a single neural network outputs both the policy $\pi_\theta(a|s)$ (actor) and a value estimate $V_\theta(s)$ (critic). The actor is trained using the policy gradient theorem with an advantage estimate, while the critic learns to approximate the state-value function through regression. The policy gradient update can be written as

$$\nabla_\theta J(\theta) = \mathbb{E}_t \left[\nabla_\theta \log \pi_\theta(a_t|s_t) A_t\right] \tag{3.14}$$

where A_t is the advantage function, typically estimated using the difference between the observed return and the predicted value function. The advantage formulation reduces variance in the gradient estimate compared to using raw returns.

In A3C, multiple worker agents run in parallel and independently collect trajectories from their environments. Each worker computes gradients using its local experiences and asynchronously applies these updates to the shared global network. This asynchronous training scheme decorrelates the data collected by different workers and stabilizes learning without the need for a replay buffer.

To further encourage exploration, A3C includes an entropy regularization term in the policy objective:

$$L = L_{policy} + \beta H(\pi(s)) \tag{3.15}$$

where $H(\pi(s))$ denotes the entropy of the policy distribution and β controls the strength of the exploration bonus.

Empirically, A3C demonstrated strong performance across a wide range of reinforcement learning benchmarks, including Atari and continuous control tasks, while significantly reducing training time through parallelization. The algorithm became one of the first scalable deep reinforcement learning methods and laid the foundation for many subsequent actor–critic algorithms [9].

Table 3.1: Comparison of PPO, SAC, and AC3 for online reinforcement learning.

Property	**PPO**	**SAC**	**A3C**
Learning type	On-policy	Off-policy	On-policy
Exploration mechanism	Policy stochasticity	Entropy maximization	Entropy regularization
Sample efficiency	Medium	High	Low
Training stability	High	High	Medium
Computational cost	Medium	Medium	Low
Parallelization	Limited	Limited	High

Offline Algorithms

IQL addresses the distribution shift problem in offline reinforcement learning by avoiding the maximization over out-of-distribution actions that appears in standard Q-learning. Instead of computing $\max_a Q(s, a)$, IQL learns a state-value function $V(s)$ directly from the dataset using *expectile regression*. This allows the algorithm to estimate the value of high-quality dataset actions without explicitly evaluating unseen actions. The Q-function is then trained using the target $r +$

$\gamma V(s')$, eliminating the need to query the Q-network for actions outside the dataset support.

Policy improvement is performed using advantage-weighted behavior cloning. Actions with higher estimated advantage are imitated more strongly, while low-advantage actions are suppressed. As a result, policy improvement occurs strictly within the dataset distribution. This greatly improves training stability compared to standard actor–critic methods and avoids the need for explicit conservative penalties. IQL has been shown to perform competitively on a range of offline RL benchmarks, particularly when datasets contain moderately good trajectories or demonstrations [6].

CQL modifies the standard Q-learning objective to learn Q-values that are conservative with respect to actions not supported by the dataset. Offline RL methods often suffer from severe overestimation of Q-values because neural networks extrapolate poorly to Out-of-Distribution (OOD) actions. When these overestimated values are used during policy improvement, the resulting policy may exploit unrealistic actions that never occur in the dataset.

CQL addresses this problem by introducing a regularization term that explicitly pushes down the Q-values of unseen actions while maintaining high Q-values for dataset actions. This effectively ensures that the expected value of a policy under the learned Q-function lower-bounds its true value. In practice, the conservative penalty reduces the likelihood that the learned policy selects actions outside the dataset support.

Because of this pessimistic value estimation, CQL is particularly robust when the dataset is noisy, narrow, or partially random. Empirical results demonstrate that CQL consistently outperforms several earlier offline RL algorithms, and behavior cloning baselines on a range of benchmark tasks [8]. The trade-off is that the conservative regularization increases computational cost and may introduce some degree of underestimation bias.

TD3 was originally introduced to address overestimation bias in deterministic actor–critic algorithms. The method uses clipped double Q-learning to reduce optimistic value estimates. Instead of relying on a single critic, TD3 maintains two Q-functions and uses the minimum of the two when constructing the Bellman target:

$$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}\big(s', \pi_{\phi'}(s')\big) \tag{3.16}$$

Using the minimum of the critics prevents the value target from introducing additional overestimation. Although this may introduce a slight underestimation bias, such bias is generally preferable because underestimated actions are less likely to be amplified through policy updates. TD3 further improves stability through the use of delayed policy updates, target networks, and target policy smoothing regularization, which reduce variance and prevent the policy from exploiting narrow peaks in the learned Q-function [3].

To adapt TD3 to the offline setting, Fujimoto and Gu (2021) introduced TD3+BC, which augments the policy objective with a behavior cloning regularization term. The policy is trained to both maximize the Q-value and remain close to actions observed in the dataset:

$$\pi = \arg\max_{\pi} \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\lambda Q(s, \pi(s)) - \|\pi(s) - a\|^2\right] \tag{3.17}$$

The hyperparameter λ controls the strength of the behavior cloning constraint. This simple modification significantly improves stability in offline settings by preventing the policy from deviating too far from the dataset distribution. Despite its simplicity, TD3+BC performs competitively with more complex offline RL algorithms on several benchmark tasks while being substantially easier to implement and computationally cheaper [2].

Table 3.2: Comparison of CQL, IQL, and TD3+BC for offline reinforcement learning.

Property	**CQL**	**IQL**	**TD3+BC**
Handles poor datasets	⋆⋆⋆	⋆⋆	⋆
Training stability	⋆⋆	⋆⋆⋆	⋆⋆
Computational cost	High	Medium	Low
Implementation complexity	Medium	Medium	Very Low
Policy improvement ability	Medium	Medium	High
Extrapolation robustness	⋆⋆⋆	⋆⋆	⋆

3.4. CFD-ANN Enriched Models

3.4.1. Other work

3.4.2. Past Work at TU Delft

The problem of solver stability has been persistent. Although multiple authors have contributed to the stability issue, the problem of long-term instability remains. In this section, the previous work of the authors at the TU Delft is explained, laying the groundwork for future progress and identifying the core issues that cause instability.

The first to try coupling a MLP with a LES solver to learn the unresolved-scale closure terms within a VMS formulation was Robijns [12]. He attempted an unconstrained approach, tasking a ANN with representing the complete unresolved-scale model. In contrast, a constrained approach has been attempted prior to Robijns (Durieux, Beekman, Kurian). Of course, hybrid models are also conceivable. The unconstrained approach will remain the basis for future work. Using the one-dimensional Burgers equation as a substitute for the Navier-Stokes equations, Robijns demonstrated that neural networks are capable of learning the underlying physical phenomena, and achieved strong *a priori* correlations between ANN predictions and DNS projected closures.

However, he performed no rigorous *a posteriori*, which led to subsequent work by the other authors.

Inserting the ANN predicted closures into an *online* simulation revealed severe stability issues. Janssens [5] set out to investigate the sources responsible for this critical stability problem. He identified two fundamental instability mechanisms: CPI, caused by ANN-induced spurious roots in the Newton loop, and LTI, caused by error accumulation. Janssens essentially shifted the research focus from "can a neural network learn the underlying physics?" to "can it remain stable in an online setting?". From this point on, it was clear that strong *a priori* correlations do not translate directly to *a posteriori* performance.

find more sources for noise-added machine learning training

A tactic to increase robustness and stability, and reduce overfitting, of an ANN is to inject the training data with artificial noise. For the VMS-ANN coupled problem, the idea was to train the neural network on "perfect" projected DNS data. This training method prevented the model from ever encountering erroneous data, leaving it unprepared for situations where the input data contains self-induced errors. Pusuluri [10] attempted this approach and found that injecting the training data with Gaussian noise indeed improved robustness and reduced sensitivity to off-manifold states encountered during online simulations. However, this came at a cost of model accuracy, and ultimately, the fundamental problem of LTI persisted.

Next, Rajampeta [11] focused on fixing the CPI problem. He introduced the LFS strategy to decouple the ANN prediction from the corrector loop. This approach addressed the CPI problem, but LTI remained, for which he introduced backscatter limiting to prevent destabilizing energy transfer from the fine to the coarse scales. Limiting enabled stable simulations but reduced physical fidelity, as backscatter is inherent.

Bettini [1] was the first to attempt the ANN-VMS formulation to a three-dimensional turbulent channel flow. His idea for addressing the LTIs was to introduce an energy-constrained training formulation. Again, Bettini achieved strong *a priori* correlations, but *a posteriori* simulations showed excessive dissipation and persistent stability issues. Indeed, energy-consistent training alone is not sufficient to guarantee physically correct long-term behavior.

Krochak [7] intended to redefine the 3D formulation, focusing on stability, solver consistency, and computational practicality. By reformulating the closure definitions, he demonstrated that modeling the continuity closure terms is particularly problematic. He evaluated three stabilization strategies: filtering of the ANN target, mini-batch sum loss, and rotational data augmentation. These improved robustness, but again, the problem of long-term instability remained.

Include Dominik in timeline

Table 3.3: Overview of past TU Delft work.

Author	Year	Problem	Approach
Robijns	2019	1D Burgers	VMM-ANN coupling feasibility
Janssens	2019	1D Burgers	*Online* analysis
Pusuluri	2020	1D Burgers	Noise-augmented training
Rajampeta	2021	1D Burgers	LFS & limiting
Bettini	2023	TCF Re_{180}	Energy-constrained
Krochak	2024	TCF	Filtering, mini-batch loss, rotational augmentation
Dominik			

4

Research Plan

4.1. Proposal

Possible to remove the accuracy part from the proposal

Research Proposal

To investigate the effectiveness of an artificial viscosity term in improving the stability of online, data-driven LES-VMS simulations applied to the one-dimensional Burgers equation, using reinforcement learning.

From the proposal, the primary research question states:

PRQ: *To what extent can an artificial viscosity term improve the stability and accuracy of VMS-ANN closure models applied to the one-dimensional Burgers equation?*

name it term or corrector?

With subsequent research questions:

RQ1. *How does the Artificial Viscosity (AV) term affect numerical stability across a range of coarse-grid resolutions compared to a no-AV baseline model?*

RQ2. *How does the artificial viscosity term affect the energy dissipation and resolved energy evolution of the VMS-ANN model compared to theoretical dissipation levels?*

RQ3. *How does the introduction of the artificial viscosity term affect the accuracy of the VMS-ANN model compared to a no-AV baseline model?*

SQ3.a *How does the artificial viscosity term affect the correlation of the resolved flow solution with DNS data, **a posteriori**?*

a posteriori can be removed as we already make the

SQ3.b *Does the artificial viscosity corrector modify the input distribution observed by the fixed closure MLP, and does this lead to observable differences in closure predictions relative to the no-AV case?*

RQ4. *Does a locally applied AV term perform better in terms of accuracy and stability compared to a global one?*

SQ4.a *Does a local model achieve better-resolved flow correlations with DNS data than the global model?*

SQ4.b *Can a local model perform longer, numerically stable simulations than the global model?*

4.2. Project Plan

Task list:

1. Create 1D Finite Element Method (FEM) solver - 1.5 weeks
   - **1.a** Develop solver code
   - **1.b** Verify solver
2. Generate training data - 1.5 weeks
   - **2.a** Run DNS
   - **2.b** Extract subgrid closures (project DNS data)
3. Train fixed MLP model - 1 weeks
4. Evaluate *a priori* correlations between fixed model and DNS - 0.5 weeks
5. Train Global Artificial Viscosity (GAV) corrector model - 2 weeks
6. Run LES-GAV model - 0.5 weeks
7. Evaluate stability and accuracy - 1 week

   Prepare Midterm - 1 week

   9 weeks → Midterm meeting
8. Train Local Artificial Viscosity (LAV) corrector model - 2 weeks
9. Run LAV model - 0.5 weeks
10. Evaluate stability and accuracy - 1 week

    Answer Research Questions
11. Run GAV, LAV across multiple grid sizes - 1 week
12. Evaluate numerical stability across multiple grid sizes - 1 week
13. Evaluate energy behavior - 1 week
14. Evaluate accuracy and correlations with DNS - 1 week

15. Evaluate differences between GAV, LAV and fixed MLP - 2 week
16. Check accuracy differences between GAV and LAV - 1 week
17. Check stability duration between GAV and LAV - 1 week

    8 weeks → If successful, attempt 3D problem.

    If applicable:
18. Train model for 3D environment - 3 weeks
19. Evaluate performance of 3D problem
20. Investigate differences between 1D and 3D problems

References

[1] Andrea Bettini. *Energy-Conservative Data-Driven Modelling for the two-scale Navier-Stokes Equations*. Tech. rep. Delft University of Technology, Aug. 2023. URL: http://repository.tudelft.nl/..

[2] Scott Fujimoto and Shixiang Shane Gu. "A Minimalist Approach to Offline Reinforcement Learning". In: (Dec. 2021). URL: http://arxiv.org/abs/2106.06860.

[3] Scott Fujimoto, Herke van Hoof, and David Meger. "Addressing Function Approximation Error in Actor-Critic Methods". In: (Oct. 2018). URL: http://arxiv.org/abs/1802.09477.

[4] Tuomas Haarnoja et al. *Soft Actor-Critic Algorithms and Applications*. 2019. arXiv: 1812.05905 [cs.LG]. URL: https://arxiv.org/abs/1812.05905.

[5] Martin Janssens. *Machine Learning of Atmospheric Turbulence in a Variational Multiscale Model*. Tech. rep. Delft University of Technology, Aug. 2019. URL: http://repository.tudelft.nl/..

[6] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. "Offline Reinforcement Learning with Implicit Q-Learning". In: (Oct. 2021). URL: http://arxiv.org/abs/2110.06169.

[7] Oleksandr Krochak. *Data-Driven Closure Modelling of the Navier-Stokes Momentum Equations*. Tech. rep. Delft University of Technology, Sept. 2024.

[8] Aviral Kumar et al. *Conservative Q-Learning for Offline Reinforcement Learning*. 2020. arXiv: 2006.04779 [cs.LG]. URL: https://arxiv.org/abs/2006.04779.

[9] Volodymyr Mnih et al. *Asynchronous Methods for Deep Reinforcement Learning*. 2016. arXiv: 1602.01783 [cs.LG]. URL: https://arxiv.org/abs/1602.01783.

[10] Abhinand Pusuluri. *Ad Noise-augmented offine training of ANN unresolved scale models*. Tech. rep. Delft University of Technology, Jan. 2020. URL: http://repository.tudelft.nl/..

[11] V S K Rajampeta. *Master Thesis Stabilization Strategies for a Variational Multiscale Method coupled with an Arti-ficial Neural Network*. Tech. rep. Delft University of Technology, Oct. 2021.

[12] Michel Robijns. *A Machine Learning Approach to Unresolved-Scale Modeling for Burgers' Equation*. Tech. rep. Delft University of Technology, Apr. 2019.

[13] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

A

MLP Extensive Explanation

This connection is mathematically presented by the sum of previous neurons multiplied by a specific weight, plus an additional bias term. Thus, for neuron j in the layer $l+1$:

change l to λ maybe, change N_L to L, and check what $\|$ means mathematically.

$$z_j^{l+1} = \sum_{i=1}^{N_l} w_{ij}^{(l)} a_i^{(l)} + b_j^{(l)} \tag{A.1}$$

Where N_L is the total amount of layers, parameters $w_{ij}^{(l)}$ are weights, parameters $b_i^{(l)}$ are biases, and $a_i^{(l)}$ is the output of neuron i in the previous layer. Afterward, a non-linear activation function is applied to the sum:

$$a_i^{(l+1)} = \sigma(z_j^{l+1}) \tag{A.2}$$

Typically, this activation function $\sigma(\cdot)$ is generally chosen to be a sigmoidal function such as the logistic sigmoid or simply the hyperbolic tangent function.

$$\sigma(z_j) = \tanh(z_j) \quad \| \quad \sigma(z_j) = (1 + \exp(-a))^{-1} \quad \| \quad \sigma(z_j) = \dots \tag{A.3}$$

To best understand the overall model architecture, it is better to collect the neurons, weights, and biases into their respective matrices $\mathbf{a}^(l)$, $\mathbf{W}^(l)$ and $\mathbf{b}^(l)$. For a single layer-to-layer connection, equation A.1 is transformed into

$$\mathbf{a}^(l+1) = \sigma\left(\mathbf{W}^(l)\mathbf{a}^(l) + \mathbf{b}^(l)\right), \quad l = 0, \dots, N_L - 1 \tag{A.4}$$

Check GPT content below

Two-Layer Multilayer Perceptron Architecture

An MLP can have an arbitrary amount of hidden layers. For simplicity, however, the explanation will continue using a two-layer (one hidden layer) MLP. Starting from the layer-wise definition in Eq. (A.4) the input to the network is given by

$$\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^{n_0}, \tag{A.5}$$

where $\mathbf{x}$ denotes the input feature vector. The hidden layer performs an affine transformation followed by a nonlinear activation function,

$$\mathbf{a}^{(1)} = \sigma^{(1)}\left(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)}\right), \tag{A.6}$$

with weight matrix $\mathbf{W}^{(0)} \in \mathbb{R}^{n_1 \times n_0}$ and bias vector $\mathbf{b}^{(0)} \in \mathbb{R}^{n_1}$. The activation function $\sigma^{(1)}(\cdot)$ is applied element-wise and introduces nonlinearity into the model.

The output layer maps the hidden-layer activations to the final network output,

$$\mathbf{a}^{(2)} = \sigma^{(2)}\left(\mathbf{W}^{(1)}\mathbf{a}^{(1)} + \mathbf{b}^{(1)}\right), \tag{A.7}$$

where $\mathbf{W}^{(1)} \in \mathbb{R}^{n_2 \times n_1}$ and $\mathbf{b}^{(1)} \in \mathbb{R}^{n_2}$. The activation function $\sigma^{(2)}(\cdot)$ is chosen according to the learning task, e.g. the identity function for regression or a sigmoid/softmax function for classification.

Combining Eqs. (A.6) and (A.7), the two-layer MLP defines the input–output mapping

$$f(\mathbf{x}) = \sigma^{(2)}\left(\mathbf{W}^{(1)}\sigma^{(1)}\left(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)}\right) + \mathbf{b}^{(1)}\right). \tag{A.8}$$

The complete set of trainable parameters of the model is given by

$$\Theta = \left\{\mathbf{W}^{(0)}, \mathbf{b}^{(0)}, \mathbf{W}^{(1)}, \mathbf{b}^{(1)}\right\}. \tag{A.9}$$

This formulation highlights that even a single hidden layer, when combined with a nonlinear activation function, yields a nonlinear mapping capable of approximating complex input–output relationships.

Training an MLP Model

Setting the parameters of the MLP model can not simply be initially chosen to yield the best performance. Instead, these parameters need to be trained. For this, the concept of *Backpropagation* is used. This is a process where the output of the model is compared to the target vector $\mathbf{t}$. After-

ward, starting from the output layer, the weights and biases are adjusted in a backwards propagating manner up until the first hidden layer, hence the name *backpropagation*. These parameters,

$$\Theta = \left\{ \mathbf{W}^{(0)}, \mathbf{b}^{(0)}, \mathbf{W}^{(1)}, \mathbf{b}^{(1)} \right\},$$

are initialized using random noise. Then, they are trained by minimizing a loss function that measures the discrepancy between the network prediction and the target output.

cite why random noise?

Let $\mathbf{y}$ denote the target output corresponding to the input $\mathbf{x}$, and let the network prediction be

$$\hat{\mathbf{y}} = f(\mathbf{x}) = \mathbf{a}^{(2)}.$$

The training objective is defined as

$$\mathcal{L} = \ell(\hat{\mathbf{y}}, \mathbf{y}), \tag{A.10}$$

where $\ell(\cdot, \cdot)$ is a suitable loss function, e.g. the mean-squared error for regression or the cross-entropy loss for classification. As the regression problem is more relevant to this project, the mean-squared error (MSE) will be the standard loss function.

$$\mathcal{L}_r = \frac{1}{2} \sum_{k=1}^{K} (y_k - t_k)^2 \tag{A.11}$$

where t_k ...

Now, the process of backpropagation begins by differentiating the loss function w.r.t. the network output. For the MSE loss, this is simply

$$\frac{\partial \mathcal{L}_r}{\partial y_k} = y_k - t_k = \delta_k \tag{A.12}$$

This quantity represents the prediction error at the output layer. As the output of the network naturally depends on the parameters of the output layer, the output error can be translated into sensitivities of the loss w.r.t. the output-layer weights and biases through the use of the chain rule. Essentially, it is asking how the output layer parameters should change in order to reduce the output error. The key idea of backpropagation is that this error can be propagated backward to earlier layers of the model. Each hidden neuron contributes to the overall loss, and it can be obtained by:

- weighting the output error by the connecting weights,
- accounting for local sensitivity of the activation function.

Through the use of the chain rule, the errors for the first hidden layer can be defined by

There are some steps in-between the first part of the equation and

still to explain briefly

- go from errors to gradients wrt to weights and biases
- from gradients, parameters are updated using a gradient-based optimization method (such as gradient descent) each parameter is updated to reduce the loss, scaled by a learning rate that controls the step size of the update
- as and end note, this process is repeated iteratively, refining the model each epoch.

$$\delta_j \equiv \frac{\partial \mathcal{L}_r}{\partial a_j} = (1 - z_j^2) \sum_{k=1}^{K} w_{kj} \delta_k \tag{A.13}$$